

Week 2 — SQL for Data Analysis: Solutions

The uploaded Week 2 sheet covers SELECT, WHERE, GROUP BY, ORDER BY, SUM/AVG/COUNT, joins, subqueries, CASE statements, and assignments involving top customers and average order values.

Important dataset note

The supplied workbook has one sales table and 200 rows. Every customer_name appears only once, so 'top customers' in this particular dataset is equivalent to ranking the individual orders by total_price. A real multi-order customer analysis would require repeated customer names or a separate customer/order structure.

1. Basic SELECT

```
-- View all rows
SELECT *
FROM sales;

-- Select useful columns
SELECT order_id, customer_name, category, total_price
FROM sales;
```

Use your actual imported table name if it differs from sales.

2. WHERE — filter rows

```
-- Orders above 20,000
SELECT order_id, customer_name, total_price
FROM sales
WHERE total_price > 20000;

-- Electronics orders
SELECT *
FROM sales
WHERE category = 'Electronics';
```

3. GROUP BY + SUM — total revenue by category

```
SELECT
    category,
    SUM(total_price) AS total_revenue
FROM sales
GROUP BY category
ORDER BY total_revenue DESC;
```

Expected totals from the supplied data: Furniture = 714,399; Grocery = 672,147; Electronics = 549,302; Clothing = 484,259.

4. AVG and COUNT

```
-- Average order value
SELECT AVG(total_price) AS average_order_value
FROM sales;

-- Number of orders
SELECT COUNT(*) AS order_count
FROM sales;

-- Average order value by category
SELECT
    category,
    AVG(total_price) AS average_order_value
FROM sales
GROUP BY category
ORDER BY average_order_value DESC;
```

Overall average order value in the supplied dataset = 12,100.535. Order count = 200.

5. ORDER BY — highest-value orders

```
SELECT order_id, customer_name, category, total_price
FROM sales
ORDER BY total_price DESC;
```

This returns the largest orders first.

6. Assignment — top customers

```
SELECT
    customer_name,
    SUM(total_price) AS total_revenue
FROM sales
GROUP BY customer_name
ORDER BY total_revenue DESC
LIMIT 10;
```

Customer	Revenue	Orders	Avg order
Lynn Garrison	47,940	1	47,940
Debbie Turner	44,990	1	44,990
Megan Charles	44,620	1	44,620
Michelle Beltran	42,471	1	42,471
Rick Sanford	41,211	1	41,211
Patrick Watson	37,680	1	37,680
Christopher Meyer	37,320	1	37,320
Todd Barnes	36,152	1	36,152
Stephanie Martinez	34,576	1	34,576
Kimberly Alvarado	31,752	1	31,752

Because each customer occurs once in this dataset, each customer's total revenue equals that customer's single order value.

7. Subquery — orders above the average order value

```
SELECT order_id, customer_name, total_price
FROM sales
WHERE total_price > (
    SELECT AVG(total_price)
    FROM sales
)
ORDER BY total_price DESC;
```

The inner query calculates the overall average; the outer query returns orders above that value.

8. CASE statement — classify order size

```
SELECT
    order_id,
    customer_name,
    total_price,
    CASE
        WHEN total_price >= 30000 THEN 'High'
        WHEN total_price >= 15000 THEN 'Medium'
        ELSE 'Low'
    END AS order_size
FROM sales
ORDER BY total_price DESC;
```

The thresholds are an example classification because the Week 2 sheet names CASE statements but does not specify thresholds.

9. JOIN — example with a customer table

```
-- This requires a second table, e.g. customers.
SELECT
```

```

    s.order_id,
    s.customer_name,
    c.customer_segment,
    s.total_price
FROM sales AS s
JOIN customers AS c
    ON s.customer_name = c.customer_name;

```

No separate customers table was supplied, so an actual join result cannot be calculated from the uploaded files. The query demonstrates the join pattern requested by the Week 2 topics.

10. Useful combined query — category summary

```

SELECT
    category,
    COUNT(*) AS order_count,
    SUM(total_price) AS total_revenue,
    AVG(total_price) AS average_order_value
FROM sales
GROUP BY category
ORDER BY total_revenue DESC;

```

This combines COUNT, SUM, AVG, GROUP BY, and ORDER BY into one common data-analysis query.

11. Data-quality checks in SQL

```

-- Missing values in important fields
SELECT
    SUM(CASE WHEN customer_name IS NULL THEN 1 ELSE 0 END) AS missing_customer,
    SUM(CASE WHEN category IS NULL THEN 1 ELSE 0 END) AS missing_category,
    SUM(CASE WHEN total_price IS NULL THEN 1 ELSE 0 END) AS missing_total_price
FROM sales;

-- Duplicate order IDs
SELECT order_id, COUNT(*) AS occurrences
FROM sales
GROUP BY order_id
HAVING COUNT(*) > 1;

```

For the supplied workbook, the corresponding Pandas checks found no missing values and no duplicate rows.

Exam/interview tip

Remember the usual SQL flow: SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT. WHERE filters rows before grouping; HAVING filters groups after aggregation.